

1. Directly using the mutex the solution might look like the following:

```
#include <iostream>
#include <chrono>
#include <random>
#include <array>
#include <thread>
#include <mutex>

int main(int argc, char const *argv[])
{
    std::array<int, 1000> numbers;

    // Seed the random number generator
    std::random_device rd;
    std::mt19937 gen(rd());
    std::uniform_int_distribution<> distrib(1, 100);

    // Fill the array with random numbers
    for (int &num : numbers)
    {
        num = distrib(gen);
    }

    // Measure the start time
    auto start = std::chrono::high_resolution_clock::now();

    // Divide the work in half
    int chunk_size = numbers.size() / 2;

    // Calculate the sum
    std::mutex sum_mutex;
    int sum = 0;

    // Create two threads
    {
        std::jthread thread1([&](int start, int end){
            for(int i = start; i < end; ++i)
            {
                sum_mutex.lock();
                sum += numbers[i];
            }
        });
    }
}
```

```

        sum_mutex.unlock();
    }
}, 0, chunk_size);
std::jthread thread2([&](int start, int end){
    for(int i = start; i < end; ++i)
    {
        sum_mutex.lock();
        sum += numbers[i];
        sum_mutex.unlock();
    }
}, chunk_size, numbers.size());
}

// Measure the end time
auto end = std::chrono::high_resolution_clock::now();

// Calculate the duration
auto duration = std::chrono::duration_cast<std::chrono::milliseconds>(end - start);

std::cout << "Sum: " << sum << std::endl;
std::cout << "Time taken: " << duration.count() << " milliseconds" << std::endl;

return 0;
}

```

2. Refactoring this to use the lock_guard.

```

#include <iostream>
#include <chrono>
#include <random>
#include <array>
#include <thread>
#include <mutex>

int main(int argc, char const *argv[])
{
    std::array<int, 1000> numbers;

    // Seed the random number generator
    std::random_device rd;
    std::mt19937 gen(rd());
    std::uniform_int_distribution<> distrib(1, 100);
}

```

```

// Fill the array with random numbers
for (int &num : numbers)
{
    num = distrib(gen);
}

// Measure the start time
auto start = std::chrono::high_resolution_clock::now();

// Divide the work in half
int chunk_size = numbers.size() / 2;

// Calculate the sum
std::mutex sum_mutex;
int sum = 0;

// Create two threads
{
    std::jthread thread1([&](int start, int end){
        for(int i = start; i < end; ++i)
        {
            std::lock_guard<std::mutex> thread1_guard(sum_mutex);
            sum += numbers[i];
        }
    }, 0, chunk_size);
    std::jthread thread2([&](int start, int end){
        for(int i = start; i < end; ++i)
        {
            std::lock_guard<std::mutex> thread1_guard(sum_mutex);
            sum += numbers[i];
        }
    }, chunk_size, numbers.size());
}

// Measure the end time
auto end = std::chrono::high_resolution_clock::now();

// Calculate the duration
auto duration = std::chrono::duration_cast<std::chrono::milliseconds>(end - start);

std::cout << "Sum: " << sum << std::endl;
std::cout << "Time taken: " << duration.count() << " milliseconds" << std::endl;

```

```
return 0;
```

```
}
```